DHAR

Bare Metal. Absolute Control.

CANON AMENDMENT A-2

AMENDMENT A-2

"Every processor and every operating system, in the first compiler, before the second begins."

DOCUMENT	Amendment A-2 — Stage 0 Targets
VERSION	1.0
DATE	October 26, 2026
STATUS	Ratified
AMENDS	Stage-alpha, Stage-beta, Stage-epsilon, Spec0, Spec1, Tooling Dhar, The Pact, the Master Index
KIND	Forward-looking specification, in the voice of the Canon
PART OF	The Canon of Dhar, Amendments
COMPANION	Amendment A-1 — Direction
AUTHOR & PROJECT LEAD	Abdullahi Baba Garba

Contents

1. Why this amendment exists 3

1.1 Why the first compiler, and not the second 3

1.2 What it costs 3

2. The decision 3

3. The targets 4

3.1 Committed targets 4

3.2 Candidate targets 5

3.3 Not targeted 5

3.4 The host 5

4. How a target is expressed in Stage 0 5

5. The system interface and the print function 6

5.1 sys 6

5.2 gen 6

6. The gate 6

7. Order, exit criteria and hand-off 7

8. What does not change 7

9. Ratification 7

1. Why this amendment exists

Stage-alpha v1.0 described a bootstrap compiler that targets x86-64 directly and only, and it placed the architecture back ends at Stage-beta, on a machine-independent intermediate representation. That was a sound plan for a bridge that is built to be crossed once. It leaves Dhar, for the whole of Stage-alpha's life, a language that produces programs for one processor family, while the people Dhar is built for use phones, single-board computers, laptops, servers and browsers on every kind of processor and system.

The project lead has decided otherwise: Dhar's first compiler produces programs for **every processor and operating system the Canon names**, and **only then** does the work of the self-hosting compiler begin. This amendment writes that decision into the Canon, lists the targets in one place, defines the test a target must pass, and records what it costs.

1.1 Why the first compiler, and not the second

- **A complete bridge.** Stage-beta (Honing Dhar) is the proof that Dhar can compile itself. It is a stronger proof when the compiler it replaces already behaves identically on every target, because the fixed-point test (Amendment A-1, S4) can then be run on each of them.
- **Reach from the first release.** A program written once in the strict subset runs on every target from the first day it can be written, instead of after the second compiler exists.
- **The browser.** The WebAssembly target is what lets anyone try Dhar in a web page with nothing to install (Tooling Dhar). That is the cheapest way for a new reader to meet the language, and it should not wait for self-hosting.
- **A single oracle.** With every back end in the first compiler, the second compiler has a complete reference to match target by target.

1.2 What it costs

Stage 0 turns each token into assembly text with no intermediate form. A new instruction set is therefore a new generator, written in the first compiler's own language (assembly), and most of it is re-done when Stage-beta writes its back ends on the intermediate representation. This amendment accepts that cost knowingly. It is bounded by one rule: **the neutral operation set and the target preludes of section 4 keep the cost of each target to one small file**, so the work done in Stage 0 is not thrown away; it becomes the test oracle for the Stage-beta back ends (section 7).

2. The decision

1. **Stage-alpha emits directly, with no intermediate representation, for every committed target** in §3.1. Stage-alpha is not complete until every committed target passes the gate of §6.
2. **An unsupported target is a fatal error**: a diagnostic, a non-zero exit status, and no output file. The compiler never falls back to another target and never produces output for a target it has not been built and tested for.

3. **Stage-beta begins when Stage-alpha is complete on every committed target.** Self-hosting work before that is preparation; it does not count toward Stage-beta's gate.
4. **After hand-off, Stage-alpha is frozen** and archived (Amendment A-1, S4). A target added after hand-off is added in Stage-beta or later, on the intermediate representation, by an amendment to §3.
5. **The print function `gen` is part of Stage-alpha** (Spec1, as amended) and behaves identically on every target (§5).

3. The targets

This is the only list of compilation targets in the Canon. Every other document links here instead of restating it (Master Index §3.4).

3.1 Committed targets

A committed target is built in Stage 0 and is part of Stage-alpha's exit criteria.

TARGET	PROGRAM FORMAT	SYSTEM INTERFACE	CALLING CONVENTION
Linux, x86-64	ELF64, static, no libc	The `syscall` instruction; number in `rax` (read 0, write 1, exit 60)	System V AMD64
Windows, x64	PE32+	A small built-in stub maps the same three numbers to the operating system's own `ReadFile`, `WriteFile` and `ExitProcess`	Windows x64
macOS, x86-64	Mach-O 64-bit	`libSystem`, the operating system's own interface library	System V AMD64
macOS, ARM64 (Apple silicon)	Mach-O 64-bit	`libSystem`	AAPCS64 (Apple variant)
Linux, ARM64	ELF64, static, no libc	`svc #0`; number in `x8` (the asm-generic table: read 63, write 64, openat 56, close 57, exit 93)	AAPCS64: arguments `x0` – `x7`, result `x0`, 16-byte stack alignment
Linux, RISC-V 64	ELF64, static, no libc	`ecall`; number in `a7` (the same table)	LP64: arguments `a0` – `a7`, result `a0`
WebAssembly with WASI (preview 1)	`.wasm` module	The WASI imports `fd_write`, `fd_read` and `proc_exit` (and the file calls of §5) over linear memory	One function per task; control flow as a loop over block numbers

On Windows and macOS the system interface is the operating system's own library, because those systems do not offer a stable raw system-call interface to programs: Apple does not promise that system-call numbers stay the same between releases, and Windows exposes its system through `kernel32` and its siblings. Calling the operating system's interface library is not a dependency on a language runtime. Principle P1 (Amendment A-1) is about the language, the compiler and the standard library, and it is met.

3.2 Candidate targets

A candidate may be promoted to §3.1 by a revision of this amendment once someone can build it and run its gate. Until then it is not part of Stage-alpha's exit criteria, and the compiler refuses it.

CANDIDATE	NOTE
FreeBSD, x86-64 and ARM64	Raw system calls are stable on FreeBSD; the work is the program format and the number table. OpenBSD restricts system calls made outside its own library and is a separate decision.
Android (Linux ARM64)	Command-line programs are the Linux ARM64 target. Packaging as an application is outside this amendment.
32-bit ARM and 32-bit x86	For old and small devices.
Bare metal and UEFI	For operating-system work, with no host system interface at all; the interface becomes the program's own responsibility.

3.3 Not targeted

iOS and iPadOS are not targeted at present: native programs may run there only inside applications signed and packaged with Apple's own toolchain, which the principle of independence (A-1, P1) does not allow Dhar to depend on. A later amendment may revisit this.

3.4 The host

Stage 0 is itself a program for Linux x86-64. It runs there and produces programs for every target in §3.1 (it is a cross-compiler). Running the compiler on another host is not required of Stage-alpha; it follows from Stage-beta, whose compiler is a Dhar program and so runs wherever Dhar programs run.

4. How a target is expressed in Stage 0

Stage 0 keeps its single token-walking emitter and has no intermediate representation. This section states the design constraint that keeps each target small; Stage-alpha v1.1 carries the detail.

- **A neutral operation set.** For the targets beyond the first, the emitter writes one neutral line per operation (load immediate, load, store, add, jump, call, and so on), with one operand slot per line, as the existing templates do.
- **One prelude per target.** Each target supplies a prelude of assembler macros that expand each neutral operation into that target's instructions. All of a target's knowledge sits in one file, readable and checkable in one place.
- **WebAssembly is structured.** It has a stack machine and structured control flow, so text templates that jump to labels do not translate directly. Its prelude uses a dispatcher form: one loop per task and a branch table over block numbers.
- **Ranges and alignment are the prelude's duty.** For example, RISC-V conditional branches reach only a short distance, so each is emitted inverted over an unconditional jump; ARM64 requires 16-byte stack alignment at calls.

- **The Strict Alpha subset does not grow** to serve a target. Where a target cannot express a construct the subset allows, the compiler refuses that construct for that target with a diagnostic; it never emits a program that behaves differently.

5. The system interface and the print function

5.1 `sys`

Programs call the system with `sys` and a number. Stage 0 accepts the Linux x86-64 numbers for the calls it supports (read, write, exit, and the file calls Spec1 lists); each target's prelude translates those numbers to its own interface, so **one source program runs unchanged on every committed target** and does the same observable thing. Spec1 (as amended) adds named forms such as `sys.write`, which carry the same meaning without numbers. Where a target cannot offer a call at all (for example, files on WebAssembly without a granted directory), a program that uses it is refused at compile time for that target, or fails with a defined error at run time; it is never silently different.

5.2 `gen`

`gen(value)` writes a value to standard output. Every implementation accepts at least these values: a string literal, a number literal, `true` or `false`, or the name of a variable of type `i32`, `str` or `bool`, whether its type was declared or inferred; Spec1 §20.2 states the full rule, and Stage-alpha §2.2 lists the further forms the bootstrap subset accepts. The call has parentheses and exactly one value. It writes exactly the text of the value: a number in decimal, a boolean as `true` or `false`, a string as its bytes, and nothing more (no added newline, no trailing zero byte). Spec1 is the home of this rule; this section states that it holds identically on every committed target and is tested on each (§6).

6. The gate

A target's gate has two levels. Both use the **same conformance battery with the same expected results on every target**: positive programs whose output or checksum is fixed; the `gen` programs, which must print exactly their text; the bounds-check programs, which must stop with the defined message and exit status; the negative programs, which must be rejected with a diagnostic and leave no output file; and the refusal of unsupported targets.

LEVEL	CONDITION	WHAT IT ALLOWS
G0, enabled	The whole battery passes when the target's output is run in an emulator or a runtime that enforces the target's rules	The compiler accepts the target on its command line; before this it refuses it
G1, complete	The whole battery passes when the target's output is run on the real operating system, on real hardware or in a real runtime for that target	The target counts toward Stage-alpha's exit criteria and may be described publicly as supported

An emulator is evidence and is recorded, but it does not pass G1. A WebAssembly runtime such as a WASI host is a real runtime for that target. A statement that Dhar supports a target names the gate and the run that passed it (Amendment A-1, §10).

7. Order, exit criteria and hand-off

Order. The order in which targets are built is a roadmap matter and lives in The Pact. The only dependencies fixed here are: the neutral operation set and the first prelude come before any other target; and the WebAssembly target comes before the in-browser playground (Tooling Dhar).

Exit criteria of Stage-alpha. Stage-alpha is complete when: every committed target has passed G1; the conformance battery, the target tables of Stage-alpha and this amendment's §3.1 agree; and the compiler still refuses every target outside §3.1.

Hand-off. On completion, Stage-alpha is frozen and archived (A-1, S4). Stage-beta begins. Its back ends are re-implemented on the intermediate representation (A-1, R1–R5), and each must pass the same battery for its target. The Stage-alpha back ends remain as the reference output against which the Stage-beta back ends are compared.

8. What does not change

- The Strict Alpha subset, apart from the addition of `gen`, and the rule that Stage-alpha is a disposable bridge with no optimiser.
- Spec1's semantics: a target changes how a program is encoded, not what it means.
- The order of the tracks in Amendment A-1, and its rule that work on the security, AI and quantum tracks begins only when Stage-beta compiles itself.

9. Ratification

This amendment takes effect when the project lead ratifies it. Each document named on the cover is then amended in the same pass.

Ratified by	Abdullahi Baba Garba, project lead
Date	October 25, 2026 (instruction to apply the amendment to every Canon document); issued as v1.0 on October 26, 2026
Notes	The committed list in §3.1, the candidates in §3.2 and the exclusion in §3.3 are the first edition of the one list of targets; promoting a candidate is a revision of this document, not a new amendment.

Revision log

Amendment A-2 — Stage 0 Targets · v1.0 · October 26, 2026 · Ratified. Contains no status: which targets are built today is in the Progress Reports, which are outside the Canon.

VERSION	DATE	CHANGE
v1.0	October 26, 2026	First edition. Records the author's decision of October 23, 2026 (the ARM64, RISC-V and WebAssembly back ends are built in Stage 0, not postponed to Stage 1) and its extension of October 25, 2026 (every processor and operating system the Canon names, with the print function gen, in Stage 0 before Stage-beta begins). Ratified on the author's instruction of October 25, 2026.